

## \* Interceptors :-

Interceptors are a powerful mechanism in Spring MVC used to perform cross-cutting concerns (like logging, authentication, or timing) around controller execution.

They allow you to execute common logic before and after controller methods without repeating code in every API.

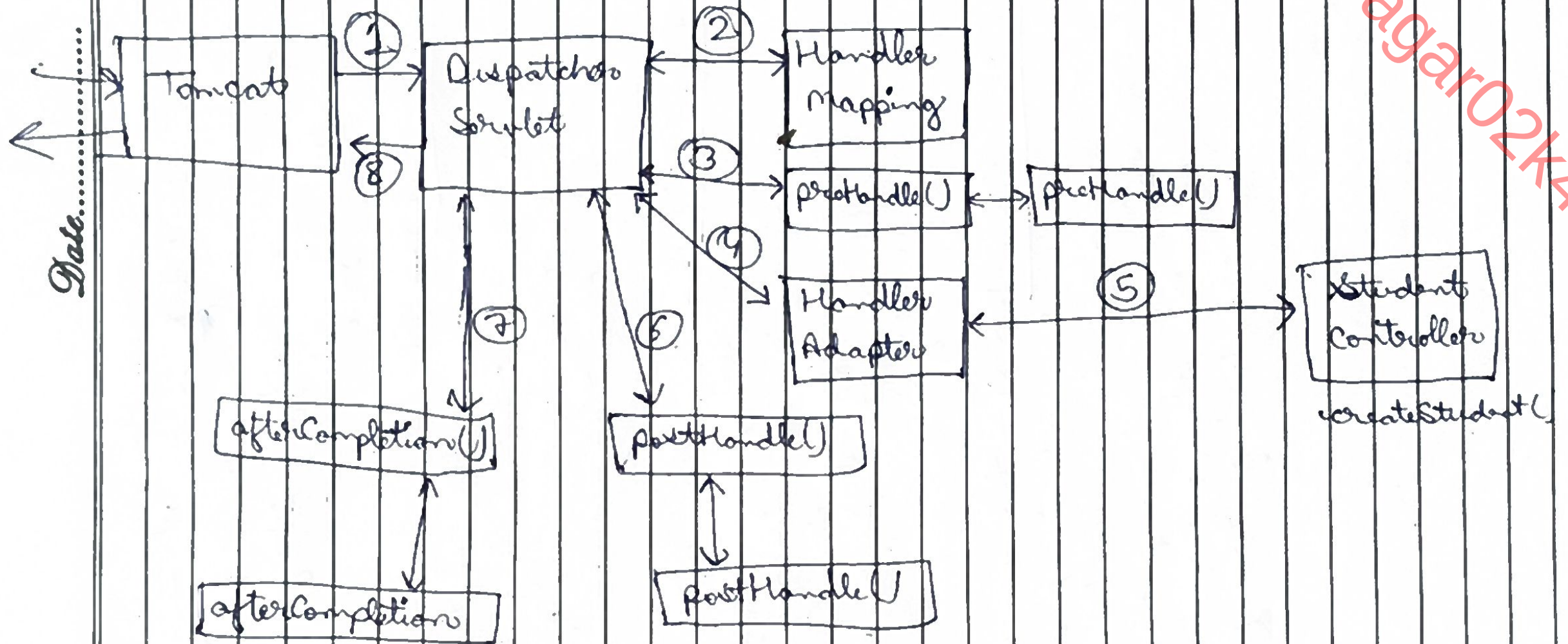
While Filters are a Servlet technology that wraps the entire request lifecycle at the Tomcat level, Interceptors are specific to Spring MVC framework and sit after 'DispatcherServlet'.

Because they operate after the DispatcherServlet has resolved the route, they have full knowledge of which controller and method will handle the request.



Sagaro2K4

Teacher's Sign



POST /api/students  
host: localhost:8080  
{  
 "id": 1  
 "name": "Sagar"  
}



To create an interceptor, you must implement the "HandlerInterceptor" interface. This interface provides three key methods that allow you to hook into diff. stages of the request lifecycle :-

- preHandle() → Executed before the request reaches the controller method. It returns a boolean: returning true allows the request to continue, while false stops the execution chain.
- postHandle() → Executed after the controller method is called but before the view is rendered. This is useful for modifying the 'ModelAndView' or adding data before the response is finalized.
- afterCompletion() → Executed once the entire request is processed and the response is sent. This is the best place for cleanup tasks or logging execution time, even if an exception occurs during the process.



Date.....

→ Registering an Interceptor

@ Configuration

```
public class WebConfig implements WebMvcConfigurer {
```

@Autowired

```
private LoginInterceptor loginInterceptor;
```

@Override

```
public void addInterceptors (InterceptorRegistry registry)
```

```
registry.addInterceptor(loginInterceptor).addPathPatterns(
```

```
"/api/**");
```

```
}
```

```
}
```

→ Most professional applications use a hybrid approach.

Filters → Reserved for global, cross-cutting concerns that need to act at the very start of a request, such as server-level request logging, or CORS config.

Interceptors → Reserved for controller-specific business logic where you need deep knowledge of the MVC routing and the target handler method.